SSL/TLS Inspection & Decryption Rollout (Netskope)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — SSL/TLS Inspection & Decryption Rollout (Netskope) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Vendor note . . . 4

Outcome . . . 4

1. Solution Overview — SSL/TLS Inspection & Decryption Rollout (Netskope) 5

Why decryption matters . . . 5

How SSL/TLS inspection works (the trust chain) . . . 5

Decryption policy and do-not-decrypt bypasses . . . 5

Privacy and compliance . . . 6

What you control . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — SSL/TLS Inspection & Decryption Rollout (Netskope) 6

2.1 Netskope tenant with SSL inspection licensed . . . 7

2.2 The inspection (forward-trust) certificate . . . 7

2.3 Intune able to deploy certificates to endpoints . . . 7

2.4 A do-not-decrypt bypass source . . . 7

2.5 A pilot group . . . 7

2.6 Privacy / compliance sign-off . . . 8

2.7 Change control and a rollback path . . . 8

Checklist . . . 8

3. Implementation Guide — SSL/TLS Inspection & Decryption Rollout (Netskope) 8

Step 1 — Assess (discovery) . . . 9

Step 2 — Create the scope groups . . . 9

Step 3 — Deploy the inspection certificate root via Intune (pilot) . . . 9

Step 4 — Build the do-not-decrypt bypass list . . . 9

Step 5 — Create the pilot decryption policy . . . 10

Step 6 — Validate certificate trust on a pilot device . . . 10

Step 7 — Validate the pilot (apps + user experience) . . . 10

Step 8 — Expand certificate trust to the fleet . . . 11

Step 9 — Expand decryption to production . . . 11

5. Testing Guide — SSL/TLS Inspection & Decryption Rollout (Netskope) 11

Test 1 — Inspection root trusted on device . . . 11

Test 2 — Decrypted sites show no certificate warning . . . 11
Test 3 — Do-not-decrypt categories are bypassed . . . 12
Test 4 — Certificate-pinned applications still work . . . 12
Test 5 — Pilot scope only (no fleet impact yet) . . . 12
Test 6 — User experience acceptable . . . 12
Test 7 — Coverage and readiness report . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — SSL/TLS Inspection & Decryption Rollout (Netskope) 12
A. Certificate warnings on decrypted sites . . . 12
B. Applications breaking after decryption is enabled . . . 13
C. Sensitive traffic being decrypted unexpectedly . . . 13
D. Pilot scope leaking to non-pilot users . . . 13
E. User-experience / performance complaints . . . 13
F. Diagnostics quick reference . . . 13
G. When to escalate . . . 14
7. Best Practices — SSL/TLS Inspection & Decryption Rollout (Netskope) 14
8. Deliverables Summary — SSL/TLS Inspection & Decryption Rollout (Netskope) 15
Documentation . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 15
Outcome . . . 16
Vendor note . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — SSL/TLS Inspection & Decryption Rollout (Netskope)

Almost all web traffic is encrypted, so a secure web gateway that cannot decrypt it is largely blind — threat protection, data-loss prevention, and content controls simply do not see the traffic that matters. SSL/TLS inspection restores that visibility. But turning on decryption carelessly breaks certificate-pinned applications, throws certificate warnings on every site, and creates privacy and compliance problems. This solution delivers a safe rollout: establish trust for the inspection certificate on every device, decrypt selectively with the right do-not-decrypt bypasses, and prove it on a pilot before enforcing.

This is a complete, deploy-it-the-repeatable-way package for rolling out Netskope SSL/TLS inspection, written for an administrator with no prior decryption-rollout experience.

What you get

- A plain-language Solution Overview of the inspection certificate trust chain, decryption policy, and do-not-decrypt bypass categories, plus the privacy/compliance considerations.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps — deploy the inspection certificate via Intune, build the decryption policy and bypass list, pilot, validate, and expand.
- Five PowerShell scripts for certificate/decryption-readiness discovery, certificate-trust validation, a coverage report, safe remediation, and scope-group targeting.
- Testing and Troubleshooting guides for a confident rollout.

What this delivers

The inspection certificate trusted silently on every device (deployed via Intune trusted-certificate profiles), a decryption policy that decrypts what you should while bypassing what you must — banking, health, and certificate-pinned apps — and a pilot-gated rollout that validates apps and user experience before the fleet is enforced.

The deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Vendor note

Netskope console internals (exact screens and menu paths) change over time and are deferred to the official Netskope documentation and your tenant; the Intune certificate steps and PowerShell scripts are exact.

Outcome

Encrypted traffic made inspectable — safely — without breaking the apps that cannot be decrypted and without exposing the traffic that should stay private, validated on a pilot before you scale.

1. Solution Overview — SSL/TLS Inspection & Decryption Rollout (Netskope)

Why decryption matters

The large majority of web traffic is now TLS-encrypted. A secure web gateway that cannot see inside that traffic cannot apply threat protection, data-loss prevention, or content/category controls to it — it is effectively blind to most of what users actually do. SSL/TLS inspection (also called TLS decryption) is what restores visibility: the gateway terminates the TLS session, inspects the plaintext, then re-encrypts to the destination.

But decryption is the single most sensitive change you can make to a secure web gateway. Turned on carelessly it breaks certificate-pinned applications, generates certificate warnings on every site, and creates privacy and compliance exposure by exposing sensitive personal traffic (banking, health, legal). The discipline that makes it safe is the same every time: establish trust for the inspection certificate on every device first, decrypt selectively with the right do-not-decrypt bypasses, and prove it on a pilot before you enforce broadly. That is the repeatable way this solution delivers.

How SSL/TLS inspection works (the trust chain)

1. The gateway performs a man-in-the-middle on the TLS session: it presents a certificate it signs on the fly for each destination, signed by an inspection (forward-trust) certificate authority.

1. For the client to trust that re-signed certificate without warnings, the inspection certificate's root (and any intermediate) must be installed in the device's trusted root store.

1. That trust is what this solution deploys via Intune trusted-certificate profiles — assign once, trusted everywhere.

1. Some applications refuse any certificate they did not expect (certificate pinning); these cannot be decrypted and must be bypassed, not forced.

Decryption policy and do-not-decrypt bypasses

A decryption policy decides, per request, whether to decrypt or pass through (do-not-decrypt). The goal is to decrypt what you should and bypass what you must:

- Decrypt the broad web you want to inspect (general browsing, unknown/uncategorized destinations, the categories where threats and data loss actually occur).
- Do-not-decrypt the categories where decryption is inappropriate or impossible:
- Banking and financial categories — privacy and regulatory sensitivity.
- Health and medical categories — sensitive personal data and compliance exposure.

- Certificate-pinned applications — they break when decrypted; pass them through.
- Other sensitive personal categories per your privacy policy (legal, government, etc.).

The bypass list is a first-class artifact in this solution, named and reused, so the same exclusions apply consistently as you expand from pilot to fleet.

Privacy and compliance

Decryption exposes the contents of user traffic to the inspection system. Treat it as a governed change: document what is decrypted and what is bypassed, exclude sensitive personal categories by default, involve privacy/legal and works-council/HR where required, and notify users per policy. A clear, documented do-not-decrypt posture is both a technical safeguard and a compliance artifact.

What you control

- Which traffic is decrypted vs. passed through (the decryption policy and its rule order).
- The do-not-decrypt bypass list (categories, domains, and pinned applications).
- The inspection certificate trust on every device (deployed and verified via Intune).
- The scope of enforcement (pilot group first, then broad), and the rollback path.

Limitations (be honest with buyers)

- Certificate-pinned applications cannot be decrypted — they must be bypassed, not forced.
- Decryption is privacy- and compliance-sensitive; sensitive personal categories should be bypassed.
- Inspection adds processing; sustained heavy traffic should be validated for user experience.
- This solution covers the inspection certificate trust chain plus Intune deployment, the decryption

policy model, and the do-not-decrypt bypass posture. Exact Netskope console screens, menu paths, and tenant-specific options change over time and are deferred to the official Netskope documentation and your Netskope tenant.

The standard deployment model

> *Assess the estate and certificate trust, configure the inspection certificate plus decryption policy and bypass list, pilot on a small group, validate apps and user experience, then expand to the fleet. The repeatable model: Assess -> Configure -> Pilot -> Validate -> Expand.*

2. Requirements Guide — SSL/TLS Inspection & Decryption Rollout (Netskope)

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-network.md`.

Named artifacts used throughout this solution (create with these exact names and reuse verbatim):

- Trusted cert profile: `Netskope Inspection Root – All Managed Devices – Pilot`

- Trusted cert profile: `Netskope Inspection Root – All Managed Devices – Production`
- Decryption policy: `SSL Inspection – Pilot Group – Pilot`
- Decryption policy: `SSL Inspection – All Users – Production`
- Bypass list: `Do-Not-Decrypt – Sensitive & Pinned – All Stages`
- Pilot group: `SSL Inspection – Pilot Users – Pilot`
- Broad group: `SSL Inspection – All Users – Production`

2.1 Netskope tenant with SSL inspection licensed

Description: An active Netskope tenant with the secure web gateway and SSL/TLS inspection capability. Why: It is the control plane that terminates and inspects TLS and enforces decryption policy. Verify: Sign in to the Netskope tenant admin console as an administrator; confirm SSL inspection is available in your subscription. Implement: Engage your Netskope account team / reseller to enable the required modules.

2.2 The inspection (forward-trust) certificate

Description: The certificate authority that signs the on-the-fly certificates the gateway presents during decryption, including its root (and any intermediate) for client trust. Why: Without the device trusting this certificate, every decrypted site throws a certificate warning. Verify: Confirm the inspection certificate exists in your Netskope tenant and you can export its root/intermediate certificate(s) for deployment. Exact export location is deferred to Netskope docs. Implement: Use the Netskope tenant's inspection certificate (default or your own enterprise-signed certificate); export the root/intermediate for Intune deployment.

2.3 Intune able to deploy certificates to endpoints

Description: Microsoft Intune managing the target devices, able to push trusted-certificate profiles. Why: The inspection certificate root must land in each device's trusted root store for silent trust. Verify: `https://intune.microsoft.com` loads; target devices are enrolled and compliant. Implement: Assign an Intune-bearing license; enroll the target devices; confirm device compliance.

2.4 A do-not-decrypt bypass source

Description: A defined set of categories, domains, and pinned applications that must NOT be decrypted (banking, health, certificate-pinned apps, and other sensitive personal categories per policy). Why: Decrypting these breaks apps or creates privacy/compliance exposure. Verify: A written do-not-decrypt list exists and is approved by privacy/security stakeholders. Implement: Build the list `Do-Not-Decrypt – Sensitive & Pinned – All Stages` from category mappings (finance, health, legal, government) plus known pinned applications; have privacy/legal review it.

2.5 A pilot group

Description: A Microsoft Entra security group of pilot users/devices for staged rollout. Why: Decryption affects live traffic and must be validated on a small group before the fleet. Verify: Entra -> Groups; `SSL Inspection – Pilot Users – Pilot` exists with representative members. Implement: Entra -> Groups -> New group -> Security; add representative pilot members; run `NSSSL-Set-ScopeTargeting.ps1` to populate it.

2.6 Privacy / compliance sign-off

Description: Documented approval that the do-not-decrypt posture and user notification meet policy. Why: Decryption exposes user traffic; sensitive personal categories and notification are governed. Verify: A privacy/compliance review of the decryption scope and bypass list is recorded. Implement: Review the decryption scope with privacy/legal and works-council/HR where required; record the approved do-not-decrypt categories and the user-notification approach.

2.7 Change control and a rollback path

Description: A change window and a documented rollback (disable the decryption policy / revert scope). Why: Inline decryption affects live traffic and must be reversible if an app or user impact appears. Verify: A rollback procedure is written and approved before enabling enforcement. Implement: Define the rollback (disable `SSL Inspection – Pilot Group – Pilot`, or remove pilot members) and a user-comms plan before enforcing.

Checklist

- Netskope tenant with SSL inspection licensed
- Inspection certificate identified and root/intermediate exportable
- Intune deploying trusted-certificate profiles to enrolled devices
- Do-not-decrypt bypass source defined (banking, health, pinned, sensitive)
- Pilot group `SSL Inspection – Pilot Users – Pilot` created
- Privacy / compliance sign-off recorded
- Change control + rollback path written and approved

3. Implementation Guide — SSL/TLS Inspection & Decryption Rollout (Netskope)

Follow these steps in order. Certificate deployment is performed in the Microsoft Intune admin center (`https://intune.microsoft.com`); the decryption policy and bypass list are configured in your Netskope tenant admin console.

> *Model: Establish certificate trust on devices first, build the decryption policy with the do-not-decrypt bypass list, steer only the pilot group, validate apps and user experience, then expand to the fleet. Pilot validates before any broad enforcement.*
>
> *Vendor note: exact Netskope console screens and menu paths change over time and are deferred to the official Netskope documentation and your tenant. Steps below name the action and the artifact; confirm the precise control in your Netskope console.*

Named artifacts (create with these exact names and reuse verbatim):

- Trusted cert profile: `Netskope Inspection Root – All Managed Devices – Pilot`

- Trusted cert profile: `Netskope Inspection Root – All Managed Devices – Production`
- Decryption policy: `SSL Inspection – Pilot Group – Pilot`
- Decryption policy: `SSL Inspection – All Users – Production`
- Bypass list: `Do-Not-Decrypt – Sensitive & Pinned – All Stages`
- Pilot group: `SSL Inspection – Pilot Users – Pilot`
- Broad group: `SSL Inspection – All Users – Production`

Step 1 — Assess (discovery)

1. Run `NSSSL-Discovery.ps1` on representative devices to report whether an inspection root is already

trusted, the OS/store state, and any obvious certificate-trust gaps. Read-only.

1. Confirm the requirements in section 2 are met; export the inspection certificate root/intermediate

from your Netskope tenant (location deferred to Netskope docs).

Step 2 — Create the scope groups

1. Entra -> Groups -> New group -> Security -> `SSL Inspection – Pilot Users – Pilot` (a few

representative users/devices). Create. Or run `NSSSL-Set-ScopeTargeting.ps1` to populate it.

1. Entra -> Groups -> New group -> Security -> `SSL Inspection – All Users – Production` (the rest of

the fleet). Create.

Step 3 — Deploy the inspection certificate root via Intune (pilot)

1. Intune -> Devices -> Configuration -> Create -> New policy.
2. Platform: Windows 10 and later. Profile type: Templates -> Trusted certificate. Create.
3. Name: `Netskope Inspection Root – All Managed Devices – Pilot`. Next.
4. Configuration settings: upload the inspection root certificate file (`.cer`); destination store:

Computer certificate store - Root. Next.

- If the inspection chain includes an intermediate, deploy it as a second Trusted certificate

profile to the Intermediate store.

1. Assignments -> Add groups -> `SSL Inspection – Pilot Users – Pilot`. Next -> Create.

Step 4 — Build the do-not-decrypt bypass list

1. In the Netskope console, create the bypass list `Do-Not-Decrypt – Sensitive & Pinned – All Stages`.
2. Add the do-not-decrypt entries (exact menu deferred to Netskope docs):

• Banking / financial categories.

• Health / medical categories.

• Certificate-pinned applications (so they are passed through, not broken).

• Other sensitive personal categories per your privacy policy (legal, government, etc.).

1. This single named list is reused by both the pilot and production decryption policies.

Step 5 — Create the pilot decryption policy

1. In the Netskope console, create the decryption (SSL inspection) policy `SSL Inspection – Pilot Group – Pilot`.

1. Source/scope: the pilot users/devices that correspond to `SSL Inspection – Pilot Users – Pilot`.

2. Action: decrypt, with the do-not-decrypt bypass list `Do-Not-Decrypt – Sensitive & Pinned – All Stages` applied ahead of the decrypt rule (bypass evaluated first).

1. Save the policy but stage it to the pilot scope only — do not apply broadly yet.

Step 6 — Validate certificate trust on a pilot device

1. On a pilot device, force a policy sync (Settings -> Accounts -> Access work or school -> Info -> Sync) so the Intune trusted-certificate profile applies.

1. Run `NSSSL-Validation.ps1` to confirm the inspection root is present in the trusted root store and the chain validates. Read-only.

1. If the root is missing, run `NSSSL-Remediation.ps1` (default `-WhatIf`) to preview the sync, then re-run with `-Execute` to request a sync, and re-validate.

Step 7 — Validate the pilot (apps + user experience)

1. On a pilot device, browse decrypted sites and confirm no certificate warnings appear.

2. Open the do-not-decrypt categories (a banking site, a health site) and confirm they are passed through (their original certificate is presented, not the inspection certificate).

1. Open known certificate-pinned applications and confirm they still work (bypassed, not broken).

2. Run `NSSSL-Report.ps1` to capture certificate-trust and decryption-readiness state across pilot devices. Complete the Testing Guide and its pass/fail gate.

Pilot gate

Do not proceed to Step 8 until the pilot gate in the Testing Guide passes: certificate trust is silent (no warnings) on pilot devices, the do-not-decrypt categories are confirmed bypassed, and known pinned applications still work. If the gate fails, use the rollback below and remediate before retrying.

> *Rollback: disable the* `SSL Inspection – Pilot Group – Pilot` *decryption policy (or remove pilot members), which immediately stops decrypting their traffic; remove the trusted-certificate profile assignment only if trust itself must be reverted. Communicate per the change plan.*

Step 8 — Expand certificate trust to the fleet

1. Create the production trusted-certificate profile `Netskope Inspection Root – All Managed Devices – Production (same root certificate, Computer store - Root) and assign it to SSL Inspection – All Users – Production`.

1. Confirm trust deploys broadly (spot-check with `NSSSL-Validation.ps1`) before enabling broad decryption — trust must lead decryption, never trail it.

Step 9 — Expand decryption to production

1. In the Netskope console, create `SSL Inspection – All Users – Production` decryption policy with the same do-not-decrypt bypass list `Do-Not-Decrypt – Sensitive & Pinned – All Stages`, scoped to `SSL Inspection – All Users – Production`.

1. Roll out in stages, re-validating with `NSSSL-Validation.ps1` and `NSSSL-Report.ps1` each stage, and chase any devices missing certificate trust before they hit decryption.

Order of operations

```
Discover -> Scope groups -> Deploy cert (pilot) -> Build bypass list -> Pilot decryption policy -> Validate trust -> Validate pilot (apps/UX) -> [pilot gate] -> Expand cert (fleet) -> Expand decryption
```

5. Testing Guide — SSL/TLS Inspection & Decryption Rollout (Netskope)

Run after the pilot decryption policy and the trusted-certificate profile apply (Implementation Steps 6-7), and again at each expansion stage.

Test 1 — Inspection root trusted on device

Do: On a pilot device run `NSSSL-Validation.ps1`, or open the certificate manager and confirm the inspection root is in Computer -> Trusted Root Certification Authorities. Expected: The inspection root (and any intermediate) is present and the chain validates.

Test 2 — Decrypted sites show no certificate warning

Do: On a pilot device, browse general/inspectable sites that are in decryption scope. Expected: Pages load cleanly; the presented certificate is the inspection-signed certificate; no certificate warnings.

Test 3 — Do-not-decrypt categories are bypassed

Do: Open a banking site and a health site (categories in `Do-Not-Decrypt – Sensitive & Pinned – All Stages`). Expected: Each site presents its original (real) certificate, not the inspection certificate — proof the bypass is taking precedence over the decrypt rule.

Test 4 — Certificate-pinned applications still work

Do: Launch known certificate-pinned applications on a pilot device. Expected: They connect and function normally (they are passed through, not decrypted/broken).

Test 5 — Pilot scope only (no fleet impact yet)

Do: On a non-pilot device, confirm traffic is not being decrypted. Expected: Non-pilot devices are unaffected until expansion (Implementation Steps 8-9).

Test 6 — User experience acceptable

Do: On pilot devices, browse normally and note page-load responsiveness during inspected traffic. Expected: No material degradation in user experience under normal browsing.

Test 7 — Coverage and readiness report

Do: Run `NSSSL-Report.ps1` across pilot devices. Expected: A per-device certificate-trust + decryption-readiness summary (root present, chain valid, last sync) with laggards flagged.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-4 pass: trust is silent, decrypted sites show no warning, do-not-decrypt categories are bypassed, and pinned apps work
Broad pass	Tests 1-3, 7 pass at each expansion stage before widening scope; no devices in scope are missing certificate trust

6. Troubleshooting Guide — SSL/TLS Inspection & Decryption Rollout (Netskope)

Use `NSSSL-Validation.ps1` / `NSSSL-Discovery.ps1` for device certificate-trust state, `NSSSL-Report.ps1` for fleet readiness, and `NSSSL-Remediation.ps1` (default `-WhatIf`) for safe fixes. Netskope console specifics are deferred to the official Netskope documentation.

A. Certificate warnings on decrypted sites

Symptom	Cause	Remediation
Every decrypted site shows a certificate warning	Inspection root not in the device trusted root store	Run `NSSSL-Validation.ps1`; if missing, run `NSSSL-Remediation.ps1` to request a sync, confirm the trusted-certificate profile is assigned to the device's group
Warning on some sites only	Missing intermediate in the chain	Deploy the inspection intermediate as a second Trusted certificate profile (Intermediate store)
Trust deployed but still warning	Device not synced / profile not applied yet	Force a sync; re-run `NSSSL-Validation.ps1` after a policy cycle

B. Applications breaking after decryption is enabled

• Certificate-pinned applications reject the inspection certificate by design. Add them to `Do-Not-Decrypt – Sensitive & Pinned – All Stages` so they are passed through, not decrypted.

• Confirm the bypass list is evaluated before the decrypt rule (rule order), or pinned traffic may be decrypted before the bypass matches.

C. Sensitive traffic being decrypted unexpectedly

• A banking/health site shows the inspection certificate instead of its own -> it is being decrypted. Confirm the category is in the do-not-decrypt list and that the bypass rule precedes the decrypt rule in the decryption policy.

D. Pilot scope leaking to non-pilot users

• Non-pilot devices are being decrypted -> the decryption policy scope or the assigned group is wider than intended. Confirm `SSL Inspection – Pilot Group – Pilot` is scoped only to `SSL Inspection – Pilot Users – Pilot`.

E. User-experience / performance complaints

• Validate page-load responsiveness on pilot devices (Testing Test 6). If degradation is real, stage expansion more slowly and review which categories truly need decryption; bypass low-value, high-volume categories where appropriate.

F. Diagnostics quick reference

```
# Inspection root present in the device trusted root store?
```

```
Get-ChildItem Cert:\LocalMachine\Root | Where-Object { $_.Subject -match 'Netskope' } | Format-List Subjec
# Validate the chain of a specific cert
$c = Get-ChildItem Cert:\LocalMachine\Root | Where-Object { $_.Subject -match 'Netskope' } | Select-Object
(New-Object System.Security.Cryptography.X509Certificates.X509Chain).Build($c)
```

G. When to escalate

If the inspection root is confirmed present and validates, the device is synced, the do-not-decrypt list and rule order are correct, and decryption still misbehaves (warnings, broken apps, wrong bypass), capture the device certificate-trust state from `NSSSL-Report.ps1` and the Netskope policy configuration, then open a case with Netskope support per the official Netskope documentation.

7. Best Practices — SSL/TLS Inspection & Decryption Rollout (Netskope)

• Trust before decrypt, always. Deploy and verify the inspection certificate root on devices before enabling decryption for them — trust must lead, never trail, or users get certificate warnings.

• Pilot first. Decryption affects live traffic; validate apps, do-not-decrypt bypasses, and user experience on a small pilot group before any broad enforcement. This is the single most important practice.

• Bypass what you must. Always do-not-decrypt banking, health, and other sensitive personal categories, and certificate-pinned applications — decrypting them breaks apps or creates privacy/compliance exposure.

• One named bypass list, reused everywhere. Maintain a single do-not-decrypt list (`Do-Not-Decrypt – Sensitive & Pinned – All Stages`) and apply it to both the pilot and production decryption policies so exclusions stay consistent as you expand.

• Order the rules correctly. Ensure the do-not-decrypt bypass is evaluated before the decrypt rule so sensitive/pinned traffic is never decrypted by accident.

• Treat decryption as a governed change. Document what is decrypted vs. bypassed, get privacy/legal (and works-council/HR where required) sign-off, and notify users per policy.

• Keep certificate trust deployment ahead of scope. Expand the trusted-certificate profile to a group before expanding the decryption policy to it.

• Keep a rollback ready. Be able to disable the decryption policy (or pull pilot members) instantly if an app or user impact appears; pre-write the comms.

• Watch coverage. Track certificate-trust and decryption-readiness with `NSSSL-Report.ps1` and chase devices missing the inspection root before they hit decryption.

• Document the configuration. Record the named profiles, policy, bypass list, and groups for audits and repeatability — assign once, repeat everywhere.

• Defer console specifics to vendor docs. Netskope console screens change; confirm the exact control in the official Netskope documentation and your tenant rather than guessing.

8. Deliverables Summary — SSL/TLS Inspection & Decryption Rollout (Netskope)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Why decryption matters, the inspection certificate trust chain, decryption policy, do-not-decrypt bypasses, privacy/compliance
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Network)	Category-wide prerequisites, published once
3. Implementation Guide	Ordered steps: deploy the inspection certificate via Intune, build decryption policy + bypass list, pilot, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`NSSSL-Discovery.ps1`	Reports OS/store state and whether an inspection root is already trusted (certificate/decryption readiness). Read-only
`NSSSL-Validation.ps1`	Confirms the inspection root is in the trusted root store and the chain validates. Read-only
`NSSSL-Report.ps1`	Fleet certificate-trust + decryption-readiness inventory via Microsoft Graph. Read-only
`NSSSL-Remediation.ps1`	Safe remediation (default `-WhatIf`); requests an MDM sync so the trusted-certificate profile re-applies
`NSSSL-Set-ScopeTargeting.ps1`	Builds/populates the pilot and production Entra scope groups via Microsoft Graph

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Encrypted traffic made inspectable — the inspection certificate trusted on every device, decryption policy enforced with the right do-not-decrypt bypasses for pinned and sensitive apps, proven on a pilot before scaling.

Vendor note

Netskope console internals are deferred to the official Netskope documentation and your tenant. Intune certificate-profile navigation and the PowerShell scripts are exact; Netskope decryption-policy and bypass-list menu paths are named at the procedural level and confirmed in your Netskope console.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

NSSSL-Discovery.ps1

Purpose: Reports OS/build, whether an inspection (forward-trust) root is already in the device trusted root store, and surfaces obvious certificate-trust gaps before a decryption rollout. Netskope decryption-policy specifics are deferred to the Netskope console (not checked here). HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSSSL-Discovery.ps1 Optional: -RootMatch 'Netskope' (substring to match the inspection root subject/issuer) EXPECTED RESULTS OS build, a list of candidate inspection roots in LocalMachine\Root with thumbprint/expiry, and a readiness note (root present / not present). .NOTES Workplace Marketplace — Network Solutions. Read-only. #> [CmdletBinding()] param([string]$RootMatch = 'Netskope') Write-Host "`n=== SSL Inspection — certificate readiness ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan $ci = Get-ComputerInfo -Property WindowsProductName,OsBuildNumber,WindowsVersion -ErrorAction SilentlyContinue Write-Host ("OS : {0} (build {1}, {2})" -f $ci.WindowsProductName, $ci.OsBuildNumber, $ci.WindowsVersion)

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSSSL-Discovery.ps1 Optional: -RootMatch 'Netskope' (substring to match the inspection root subject/issuer) EXPECTED RESULTS OS build, a list of candidate inspection roots in LocalMachine\Root with thumbprint/expiry, and a readiness note (root present / not present). .NOTES Workplace Marketplace — Network Solutions. Read-only. #> [CmdletBinding()] param([string]$RootMatch = 'Netskope') Write-Host "`n=== SSL Inspection — certificate readiness ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan $ci = Get-ComputerInfo -Property WindowsProductName,OsBuildNumber,WindowsVersion -ErrorAction SilentlyContinue Write-Host ("OS : {0} (build {1}, {2})" -f $ci.WindowsProductName, $ci.OsBuildNumber, $ci.WindowsVersion)

NSSSL-Remediation.ps1

Purpose: When NSSSL-Validation.ps1 reports the inspection root is missing, this triggers an Intune/MDM sync so the assigned Trusted certificate profile re-applies to the device. Read-only by default (-WhatIf): it previews the action and changes nothing until -Execute is passed. It never installs a certificate directly — trust is governed by the Intune profile. Netskope decryption-policy changes are made in the Netskope console (not by this script). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSSSL-Remediation.ps1 Apply : powershell -ExecutionPolicy Bypass -File .\NSSSL-Remediation.ps1 -Execute EXPECTED RESULTS In preview: prints what would run. With -Execute: an MDM sync is requested; re-run NSSSL-Validation.ps1 after a policy cycle. .NOTES Workplace Marketplace — Network Solutions. Default -WhatIf (no changes without -Execute). #> [CmdletBinding()] param([switch]$Execute) $WhatIf = -not $Execute Write-Host "=== SSL Inspection Remediation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan Write-Host ("Mode: {0}" -f ($(if($WhatIf){'WHATIF (preview only — pass -Execute to apply)'}else{'EXECUTE'}))) -ForegroundColor ($(if($WhatIf){'Yellow'}else{'Green'}))

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSSSL-Remediation.ps1 Apply : powershell -ExecutionPolicy Bypass -File .\NSSSL-Remediation.ps1 -Execute EXPECTED RESULTS In preview: prints what would run. With -Execute: an MDM sync is requested; re-run NSSSL-Validation.ps1 after a policy cycle. .NOTES Workplace Marketplace — Network Solutions. Default -WhatIf (no changes without -Execute). #> [CmdletBinding()] param([switch]$Execute) $WhatIf = -not $Execute Write-Host "=== SSL Inspection Remediation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan Write-Host ("Mode: {0}" -f ($(if($WhatIf){'WHATIF (preview only — pass -Execute to apply)'}else{'EXECUTE'})))

-ForegroundColor ($(if($WhatIf){'Yellow'}else{'Green'}))

NSSSL-Report.ps1

Purpose: Reports, across managed Windows devices in Intune, the deployment state of the trusted inspection-certificate profile and basic device readiness (last sync), so you can confirm certificate trust precedes decryption and chase laggards before they hit a decrypt policy. Netskope decryption-policy coverage is in the Netskope console (deferred). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\NSSSL-Report.ps1 -ProfileNameMatch 'Netskope Inspection Root' -CsvPath C:\Temp\nsssl.csv Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Per-device deployment state (Succeeded/Error/Pending) for the matching trusted-cert profile(s) and a breakdown of how many devices have trust deployed; optional per-device CSV. .NOTES Workplace Marketplace — Network Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$ProfileNameMatch = 'Netskope Inspection Root', [string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementConfiguration.Read.All','DeviceManagementManagedDevices.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\NSSSL-Report.ps1 -ProfileNameMatch 'Netskope Inspection Root' -CsvPath C:\Temp\nsssl.csv Requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Per-device deployment state (Succeeded/Error/Pending) for the matching trusted-cert profile(s) and a breakdown of how many devices have trust deployed; optional per-device CSV. .NOTES Workplace Marketplace — Network Solutions. Read-only against Graph. #> [CmdletBinding()] param([string]$ProfileNameMatch = 'Netskope Inspection Root', [string]$CsvPath) if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementConfiguration.Read.All','DeviceManagementManagedDevices.Read.All' -NoWelcome

NSSSL-Set-ScopeTargeting.ps1

Purpose: Ensures the Entra security groups used to scope SSL inspection exist and, optionally, adds members to the pilot group — so certificate trust and the decryption policy can be assigned to a controlled scope first. This script manages GROUPS only; the Netskope decryption policy and bypass list are configured in the Netskope console (deferred). HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\NSSSL-Set-ScopeTargeting.ps1 Create groups + add pilot members: powershell -ExecutionPolicy Bypass -File .\NSSSL-Set-ScopeTargeting.ps1 -Execute -PilotMemberUpns user1@contoso.com,user2@contoso.com Requires Microsoft Graph SDK and Group.ReadWrite.All + User.Read.All. EXPECTED RESULTS In preview: prints whether each group exists and what would be created/added. With -Execute: creates any missing group and adds the supplied UPNs to the pilot group. .NOTES Workplace Marketplace — Network Solutions. Default preview (no changes without -Execute). #> [CmdletBinding()] param([string]$PilotGroupName = 'SSL Inspection – Pilot Users – Pilot', [string]$ProductionGroupName = 'SSL Inspection – All Users – Production', [string[]]$PilotMemberUpns = @(), [switch]$Execute) $WhatIf = -not $Execute if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome Write-Host ("Mode: {0}" -f

($(if($WhatIf){'PREVIEW (pass -Execute to apply)'}else{'EXECUTE'}))) -ForegroundColor ($(if($WhatIf){'Yellow'}else{'Green'}))

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\NSSSL-Set-ScopeTargeting.ps1 Create groups + add pilot members: powershell -ExecutionPolicy Bypass -File .\NSSSL-Set-ScopeTargeting.ps1 -Execute -PilotMemberUpns user1@contoso.com,user2@contoso.com Requires Microsoft Graph SDK and Group.ReadWrite.All + User.Read.All. EXPECTED RESULTS In preview: prints whether each group exists and what would be created/added. With -Execute: creates any missing group and adds the supplied UPNs to the pilot group. .NOTES Workplace Marketplace — Network Solutions. Default preview (no changes without -Execute). #> [CmdletBinding()] param([string]$PilotGroupName = 'SSL Inspection – Pilot Users – Pilot', [string]$ProductionGroupName = 'SSL Inspection – All Users – Production', [string[]]$PilotMemberUpns = @(), [switch]$Execute) $WhatIf = -not $Execute if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome Write-Host ("Mode: {0}" -f ($(if($WhatIf){'PREVIEW (pass -Execute to apply)'}else{'EXECUTE'}))) -ForegroundColor ($(if($WhatIf){'Yellow'}else{'Green'}))

NSSSL-Validation.ps1

Purpose: Verifies the SSL inspection (forward-trust) root is present in the device trusted root store and that its certificate chain builds without error — the prerequisite for warning-free decryption. Netskope decryption-policy state is deferred to the Netskope console. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\NSSSL-Validation.ps1 Optional: -RootMatch 'Netskope' EXPECTED RESULTS "INSPECTION TRUST OK" when a matching root is present and the chain validates (exit 0); otherwise a remediation hint (exit 1). .NOTES Workplace Marketplace — Network Solutions. Read-only. #> [CmdletBinding()] param([string]$RootMatch = 'Netskope') Write-Host "=== SSL Inspection Validation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan $root = Get-ChildItem Cert:\LocalMachine\Root -ErrorAction SilentlyContinue | Where-Object { $_.Subject -match $RootMatch -or $_.Issuer -match $RootMatch } | Select-Object -First 1

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\NSSSL-Validation.ps1 Optional: -RootMatch 'Netskope' EXPECTED RESULTS "INSPECTION TRUST OK" when a matching root is present and the chain validates (exit 0); otherwise a remediation hint (exit 1). .NOTES Workplace Marketplace — Network Solutions. Read-only. #> [CmdletBinding()] param([string]$RootMatch = 'Netskope') Write-Host "=== SSL Inspection Validation — $($env:COMPUTERNAME) ===" -ForegroundColor Cyan $root = Get-ChildItem Cert:\LocalMachine\Root -ErrorAction SilentlyContinue | Where-Object { $_.Subject -match $RootMatch -or $_.Issuer -match $RootMatch } | Select-Object -First 1